11-WEEK ENGINEERING BLUEPRINT • TECHNICAL TRANSLATION

CyberPulse Security Awareness & Quiz Platform

Humanoid Architectural Breakdown & Stakeholder Implementation Plan

Next.js 14 + Supabase
Target Host: Vercel Cloud

Meet Proto: Your Humanoid Cyber-Mentor

"Welcome! CyberPulse is a security learning platform where students master cybersecurity topics, take timed quizzes, and climb leaderboards. **We are NOT rebuilding this codebase from scratch!** The foundation, UI screens, quizzes, and admin routes are already written in the repository. This 11-week proposal is a surgical hardening mission: plugging data leaks, establishing server-side scoring, automating test coverage, and locking down permissions for a safe, shared production rollout."

1. The Core Proposal Explained Through 3 Human Analogies

PLAIN-ENGLISH FUNDAMENTALS

ANALOGY #1: INTEGRITY

The Restaurant Bill

A customer's phone is like a diner looking at a menu. You would never let a customer write their own total bill at checkout!

Week 5 Solution: The browser only sends selected choices. The server re-grades answers from trusted DB records so nobody cheats.

ANALOGY #2: SECRETS

The Bank Master Key

Supabase provides a "Service Role Key" which is a skeleton key that unlocks every vault in the bank without restriction.

Week 2 & 9 Solution: That master key stays locked strictly in Vercel server environment variables. Zero client leaks.

ANALOGY #3: PRIVACY

Private Lockers (RLS)

Every student has a gym locker. User A must never be able to open User B's locker, even if they guess the locker number.

Week 4 & 10 Solution: Postgres Row-Level Security (RLS) enforces that users only access their own quiz history & avatars.

2. Technical Jargon Decoded into Business & User Benefits

CROSS-REFERENCE TABLE

PROPOSAL TECHNICAL TERM	WHAT PROTO EXPLAINS (HUMAN MEANING)	CONCRETE IMPACT ON CYBERPULSE
`Next.js 14 App Router on Vercel`	The modern cloud web server hosting the website globally.	Instant page loads across mobile & desktop, zero server maintenance headaches, and continuous preview deploys.
`Supabase Auth, PostgreSQL & Storage`	The unified database engine holding user accounts, quiz questions, and profile photos.	One single source of truth: eliminates orphaned attempts, handles user signups securely, and scales seamlessly.
`PostgreSQL Row-Level Security (RLS)`	Security guards stationed directly at the database tables.	Blocks data breaches at the database layer; students cannot snoop on classmates' scores even with raw API queries.
`Server-Side Scoring Integrity`	The teacher calculates the grade in the classroom, not the student.	Protects leaderboard integrity; prevents devtools tampering where users fake a 100% score via client JSON packets.
`Playwright & Vitest Quality Gates`	Robots taking 500 quizzes and testing every button before release.	Catches regressions instantly; guarantees that updates don't accidentally break logins or reset quiz progress.

Proto's Golden Architecture Rule:
"Never trust anything coming from a user's web browser! The browser displays the interface, but the server does the authentication, score validation, and secret keeping."

ARCHITECTURE & TRUST BOUNDARIES

Logical System Architecture & Security Tiers

How Browser, Vercel Server, and Supabase Database Cooperate Safely

Zero Client Secret Policy
Figure 1 Detailed Walkthrough

The Three-Tier Isolation Model

Red Line = The Hard Security Boundary

Tier 1: Browser Client

UNTRUSTED ENVIRONMENT

- ✓ Responsive Next.js 14 UI
- ✓ User Login & Signup Forms
- ✓ Interactive Quiz Flow
- ✓ Profile & Leaderboard Views

🔑 **Key Used:** Supabase Anon Key (Public only)

→

Tier 2: Vercel Server

TRUSTED GATEKEEPER

- ✓ Verifies Supabase User Session
- ✓ Computes Authoritative Scores
- ✓ Enforces Admin Role Verification
- ✓ Input Validation via Schema (Zod)

🔒 **Key Kept:** Secret Service Role Key (Server only)

→

Tier 3: Supabase Vault

SHARED PERSISTENCE

- ✓ PostgreSQL Relational Tables
- ✓ Strict Row-Level Security (RLS)
- ✓ Supabase Auth Tokens & Sessions
- ✓ Media Buckets (User Avatars)

🛡️ **Rule:** User isolation enforced in SQL

3. Implementation Plan: Weeks 1 to 5 (Audit, Setup, Security & Scoring)

PHASE 1 & PHASE 2

W1 **Repository Audit & Baseline Discovery** — PHASE 1: BASELINE

Map all existing Next.js routes, components, Supabase client calls, SQL scripts, and document current defects.

Deliverables: Architecture inventory, feature checklist, setup notes, risk register, baseline test scenarios.

W2 **Local Setup & Environment Configuration** — PHASE 1: ENVIRONMENT

Standardize Node/npm versions, environment variable names, create clean .env.example, and verify reproducible local builds.

Deliverables: Reproducible local setup guide, validated environment template, build report.

W3 **Authentication & Authorization Review** — PHASE 2: AUTH HARDENING

Trace signup, login, session middleware, and admin role determination. Construct a formal user/admin permission matrix.

Deliverables: Auth flow diagram, role/permission matrix, prioritized security findings, approved remediation plan.

W4 **Supabase Schema, RLS & Data Integrity** — PHASE 2: DATABASE

Review database tables, foreign keys, indexes, and write strict RLS policies ensuring users can only read their own data.

Deliverables: Database & RLS review, migration scripts, repeatable seed procedures, cross-user policy test cases.

W5 **Quiz Workflow & Scoring Integrity** — PHASE 2: ANTI-CHEAT

Move score calculation from untrusted client to server API handlers. Compare answers against trusted database keys.

Deliverables: Verified quiz submission workflow, server scoring test suite, API payload validation, edge-case documentation.

IMPLEMENTATION ROADMAP CONTINUED

Implementation Plan: Weeks 6 to 11

Admin Operations, Automation, Cloud Staging, Security QA & Handover

Phases 3, 4 & 5
Towards Production Live

W6 **Admin Content Management & Safe Operations** — PHASE 3: ADMIN

Harden quiz/question CRUD screens and AI question-generation endpoints with strict server-side admin role verification.

Deliverables: Hardened admin APIs, CRUD regression tests, admin workflow verification report.

W7 **Progress, Achievements, Leaderboard & Profile** — PHASE 3: ENGAGEMENT

Validate calculations for streaks, achievements, competencies, and secure avatar uploads in Supabase Storage.

Deliverables: Regression-tested user dashboards, avatar access checks, query performance improvements.

W8 **Automated Testing & Quality Gates** — PHASE 4: AUTOMATION

Implement Vitest/Jest unit tests for utilities and Playwright end-to-end tests for login, quiz taking, and admin security.

Deliverables: Automated test suite, CI build verification checklist, test coverage report on critical user flows.

W9 **Deployment & Production Configuration** — PHASE 4: CLOUD STAGING

Configure Vercel project with separate Production/Preview env variables, verify Supabase redirect URLs, deploy to staging.

Deliverables: Live staging deployment, deployment runbook, environment checklist, smoke-test evidence.

W10 **Security, Performance & Acceptance Testing** — PHASE 4: HARDENING

Perform cross-user isolation penetration checks, API abuse fuzzing, mobile responsive audits, and fix release blockers.

Deliverables: Security/acceptance test report, prioritized defect remediation, official Release Candidate (RC).

W11 **Production Release, Documentation & Handover** — PHASE 5: LIVE LAUNCH 🚀

Deploy approved release to production Vercel instance, verify smoke tests, confirm rollback procedures, and hand over source ownership.

Deliverables: Live production deployment, final technical docs, admin user guide, handover checklist, final project report.

4. Phase Summary & Effort Distribution

77 CALENDAR DAYS

PHASE	WEEKS	PRIMARY FOCUS AREA	KEY STAKEHOLDER OUTCOME
Phase 1: Foundation	Weeks 1 – 2	Repo audit, environment configs, local setup standardization.	A clean, buildable project that any engineer can run locally in minutes.
Phase 2: Security Core	Weeks 3 – 5	Auth tracing, Postgres RLS lockdown, server-side score calculation.	An uncheatable quiz engine and leak-proof database isolation.
Phase 3: Operations	Weeks 6 – 7	Admin CRUD, AI question generation security, dashboard metrics.	Admins manage quiz content safely without accidentally corrupting data.
Phase 4: QA & Cloud	Weeks 8 – 10	Automated unit & E2E tests, Vercel staging deploy, penetration testing.	Zero surprise crashes; verified staging instance connected to Supabase.
Phase 5: Production	Week 11	Production release, smoke tests, rollback runbook, client handover.	Full client ownership of live, documented, production-grade platform.

RISK GOVERNANCE & FINAL ACCEPTANCE

Technical Risk Mitigations & Acceptance Criteria

Ensuring Long-Term Security, Stability, and Measurable Project Success

Sections 6 & 7
Sign-Off Readiness

5. The 6 Critical Risks & How We Neutralize Them

PROACTIVE RISK MANAGEMENT

CRITICAL RISK #1

Privileged Key Exposure

If the Supabase Service Role Key leaks into client JavaScript bundles, any user can bypass RLS and delete database tables.

Mitigation: Key is restricted to Vercel server environment variables; client build scanners verify no leak in bundles.

HIGH RISK #2

Admin Permission Mistakes

Hiding admin links in the UI does not stop malicious users from calling the admin API endpoints directly.

Mitigation: Every privileged API operation verifies user session and admin role on the server before executing.

HIGH RISK #3

RLS Policy Regressions

Schema changes can accidentally open cross-user access, allowing students to view other students' attempts or profile data.

Mitigation: Automated penetration test cases run in CI to verify that cross-user queries return 0 rows.

HIGH RISK #4

Scoring Manipulation

Users tamper with request payloads to award themselves 100% scores and counterfeit certificates or badges.

Mitigation: Client inputs are treated as untrusted; server recomputes results against authoritative question answer keys.

MEDIUM RISK #5

Deployment Config Mismatch

Preview and Production Vercel environments using mismatched redirect URLs, causing broken auth loops.

Mitigation: Maintain an environment checklist, explicit allowed origins in Supabase, and staging smoke tests.

PROJECT RISK #6

Scope Creep Trap

Attempting to invent brand-new features before stabilizing and testing what already exists in the repository.

Mitigation: Prioritize existing routes and workflows; optional tools (e.g. Sentry) only added with explicit approval.

6. Final Acceptance Criteria (The Definition of Done)

SUCCESS CHECKLIST

- ✓ **Existing Workflows Intact:** All existing quiz browsing, taking, review, leaderboards, and admin CRUD remain fully operational.
- ✓ **Strict RLS Enforcement:** Authenticated users can access only their own records; cross-user queries are denied by PostgreSQL.
- ✓ **Server-Side Admin Gates:** All admin operations reject non-admin users at the server/API layer with zero client trust.
- ✓ **Server-Calculated Scores:** Quiz attempts are calculated and stored authoritatively on the server.
- ✓ **Zero Secret Leakage:** Supabase Service Role Key is kept exclusively on Vercel server environment variables.
- ✓ **Automated Quality Gates:** Critical paths are covered by automated unit and E2E test suites with passing CI gates.
- ✓ **Turnkey Handover:** Comprehensive documentation includes local setup, deployment runbooks, and admin user guides.

CyberPulse 11-Week Engineering Engagement
Baseline: Existing Next.js 14 + Supabase Repository • Host: Vercel Cloud

APPROVED FOR EXECUTION